

Lessons from 6 Months of Building Agents

November 19, 2025

1 Introduction

Building AI agents is a rapidly evolving field filled with hype. This report synthesizes critical lessons learned over six months of practical experience, moving beyond simple demos to understanding what it takes to build robust, production-ready systems.

2 Hard Truths about AI Agents

2.1 Demos vs. Production Reality

Most AI agent demos seen online (LinkedIn, YouTube, blogs) are proof-of-concepts or MVPs (Minimum Viable Products). They are designed to showcase possibilities and visual workflows, not to be immediate, production-ready solutions for businesses.

- **Fragility:** Demos often break when exposed to edge cases, real-world data, or high user volume.
- **Enterprise Struggle:** Even tech giants like Apple and Google face delays and reliability issues with AI deployments, highlighting the difficulty of creating stable production systems.
- **Goal of Demos:** Their primary value is educational—teaching fundamentals and visual understanding of tool usage—rather than providing a “copy-paste” business solution.

2.2 AI Agents vs. AI Workflows

There is a crucial distinction between these two concepts, often conflated in the industry.

- **AI Workflow (Deterministic):** A sequential, pre-defined process. Data moves linearly from Step A to Step B to Step C. It is predictable, easier to debug, and more consistent. Most business problems can be solved with AI-enhanced workflows rather than full agents.
- **AI Agent (Non-Deterministic):** A system where the AI decides the path. It autonomously selects which tools to use and in what order based on the input. This is necessary for dynamic decision-making but introduces variability and complexity.

3 The 7 Key Lessons

3.1 1. Build Workflows First

Start with the simplest approach. Don't force an agent into a process that can be solved with a rule-based or linear AI workflow.

- **Assess Needs:** Does the process require dynamic reasoning? If not (e.g., routing emails based on simple keywords), use logic rules.
- **Scale Up:** Start with a workflow. Only upgrade to an agent when you need the system to autonomously decide steps, handle complex reasoning, or manage dynamic token usage.

3.2 2. Wireframe Before Building

Avoid jumping straight into the builder. Planning is essential to prevent "spaghetti code" workflows.

- **Map it Out:** Use tools like Excalidraw or pen and paper to visualize the data flow, triggers, and actions.
- **Ask Questions:** Define triggers, data movement, required AI decisions, and necessary integrations upfront.
- **Blueprint:** Treat the wireframe as an instruction manual. It aligns expectations (with clients or yourself) and makes the actual building process much faster and cleaner.

3.3 3. Context is Everything

An AI model is only as good as the context it is given. Without domain-specific knowledge, even the best model (like GPT-4) will fail to be a "superstar employee."

- **Preloaded Knowledge (System Prompt):** Define the agent's role, responsibilities, and rules clearly (e.g., "You are a legal assistant...").
- **User-Specific Context (Memory):** Store past interactions so the agent remembers user preferences and doesn't ask repetitive questions.
- **Real-Time Context (RAG/Tools):** Give the agent access to external data (via APIs or vector databases) to retrieve up-to-date information it wasn't trained on.

3.4 4. When NOT to Use Vector Databases

Vector databases are powerful for unstructured text (semantic search), but they are not a silver bullet.

- **Structured Data:** For business data like sales records, customer IDs, or inventory counts, use **Relational Databases (SQL)**. They are faster and more accurate for filtering and exact matches.
- **Unstructured Data:** Use Vector Databases for searching through large volumes of text (PDFs, documentation) where meaning and context are more important than exact keyword matches.

3.5 5. Reactive Prompting is Superior

Don't try to write a perfect, massive prompt upfront ("Proactive Prompting"). It makes debugging impossible.

- **The Reactive Method:** Start with a blank or minimal prompt. Add one tool. Test it. Observe the failure.
- **Fix and Repeat:** Add a specific line to the prompt to fix *that specific error*. Repeat this loop.
- **Hard Prompting:** If an agent consistently fails a specific task, "hard code" a rule or example into the prompt to force the correct behavior (e.g., "Always get the email address before sending").

3.6 6. Scaling is a Challenge

A system that works perfectly for one user often breaks at scale.

- **Performance Degradation:** As users and data increase, you may face more hallucinations, slower retrieval times, and higher latency.
- **Scale Vertically First:** Perfect the system for one specific domain or use case (e.g., just customer support) before trying to expand horizontally to other departments (sales, HR, etc.).
- **Optimization:** Scaling requires implementing strict guardrails, better data segmentation, and potentially caching strategies.

3.7 7. No-Code Tools Have Limits

Platforms like n8n are incredible for speed, modularity, and accessibility, but they have ceilings.

- **Great For:** MVPs, internal tools, rule-based automation, and orchestrating AI services.
- **Struggles With:** Enterprise-scale applications with thousands of concurrent users, complex custom authentication flows, or extremely heavy data processing.
- **The Hybrid Solution:** The best approach for complex builds often involves a mix: using no-code for orchestration and custom code (Python/JavaScript) for specific, heavy-lifting tasks or complex logic.